FECAP
Ciência da Computação - 6° Semestre

Teoria da Computação e Linguagens Formais

Entrega 2

Integrantes:

Bernardo Seijas Cavalcante
Carlos Roberto Santos Latorre
Felipe Wakasa Klabunde
Stephany Aliyah Guimarães Eurípedes de Paula

Introdução

No contexto do projeto Smart Tennis Arena, o TennisSync consiste em um treinador interativo de tênis que utiliza visão computacional e gamificação para auxiliar o jogador na execução correta dos movimentos. A aplicação apresenta um movimento de referência na tela e permite que o usuário inicie a captura por meio de um botão na interface. Após a execução, os dados obtidos pela câmera e processados com auxílio do MediaPipe são comparados com os parâmetros definidos para o exercício, permitindo identificar possíveis erros e apresentar uma pontuação, além de orientações corretivas por meio de feedback visual e por voz.

Esta segunda entrega tem como objetivo atualizar a modelagem desenvolvida anteriormente e documentar formalmente os eventos, as regras de funcionamento, as condições das operações e o modelo computacional adotado, incluindo uma análise de decidibilidade das principais decisões realizadas durante o treinamento.

1. Atualização da Máquina de Estados

A máquina de estados desenvolvida na primeira entrega foi revisada para acompanhar as alterações realizadas durante a evolução do projeto. A principal modificação corresponde à retirada do pedal, anteriormente utilizado para iniciar a captura do movimento, que foi substituído por um botão disponível na interface.

Essa alteração modifica o evento responsável pela transição entre o estado de espera e o estado de captura, mas não exige a criação de novos estados. A estrutura definida anteriormente continua representando as principais etapas de funcionamento, desde a inicialização dos componentes até o encerramento da sessão de treinamento.

1.1 Estados do Sistema

Para representar o funcionamento da aplicação, foram definidos os principais estados pelos quais ela pode passar durante uma sessão de treinamento. Cada estado corresponde a uma etapa específica, desde a inicialização dos componentes até a análise do movimento, apresentação do feedback e encerramento da sessão.

Os estados identificados foram mantidos conforme a modelagem da primeira entrega, considerando as alterações realizadas no projeto, principalmente a substituição do pedal por um botão na interface para iniciar a captura.

- **q0 – Inicializando:** é o estado inicial. Nessa etapa, são iniciados o software, a câmera, o MediaPipe e a comunicação com o Arduino.
- **q1 – Pronto:** indica que a inicialização terminou corretamente e que os recursos necessários estão disponíveis para iniciar o treinamento.
- **q2 – Demonstração de Movimento:** a tela apresenta ao jogador o movimento de referência que deverá ser executado.
- **q3 – Aguardando:** após a demonstração, a aplicação aguarda o acionamento do botão na tela para iniciar a captura. Nesse estado, a sessão também pode ser pausada ou encerrada.

- **q4 – Captura:** a câmera registra a execução do jogador e o MediaPipe identifica os pontos de referência do corpo.
- **q5 – Análise:** os dados capturados são processados e comparados com os parâmetros definidos para o movimento esperado.
- **q6 – Correto:** indica que a execução foi considerada correta de acordo com os critérios estabelecidos. A partir desse estado, o treinamento pode avançar para o próximo movimento ou ser encerrado.
- **q7 – Erro de Movimento:** indica que foi identificado um erro na execução do movimento. Esse estado representa uma tentativa incorreta do jogador, e não uma falha técnica.
- **q8 – Feedback:** são apresentadas ao jogador as orientações necessárias para corrigir o movimento, utilizando a tela, os LEDs e o retorno por voz.
- **q9 – Pausado:** representa uma interrupção temporária da sessão. Ao retomar, o treinamento retorna à demonstração do movimento.
- **q10 – Encerrado:** representa o estado final da sessão de treinamento.
- **qE – Falha:** representa uma falha técnica, como problemas na câmera, perda de comunicação com o Arduino ou erros críticos de software ou hardware.

A distinção entre q7 (Erro de Movimento) e qE (Falha) permanece importante para a modelagem. O primeiro representa uma tentativa executada fora dos parâmetros definidos, enquanto o segundo indica um problema técnico que compromete o funcionamento da aplicação.

1.2 Diagrama de Estados Atualizado

O diagrama atualizado apresenta o fluxo de funcionamento do TennisSync, considerando os estados e as transições responsáveis pelo controle do treinamento. A estrutura definida na primeira entrega foi preservada, tendo como principal alteração a substituição do pedal por um botão na interface para iniciar a captura do movimento. A representação contempla as etapas de inicialização, demonstração, captura, análise, apresentação dos resultados, feedback, pausa, encerramento e tratamento de falhas técnicas.

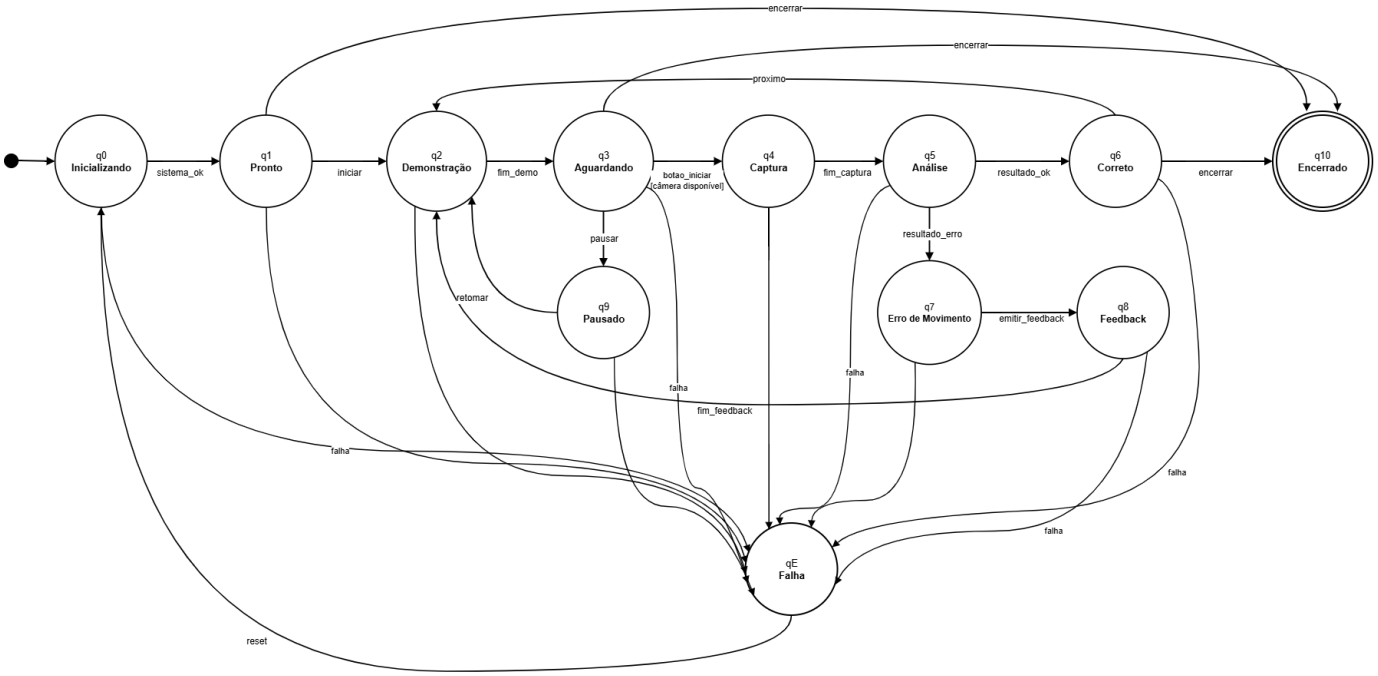


Figura 1 — Diagrama de Estados Atualizado do TennisSync

O fluxo principal começa em q0 (Inicializando). Após a conclusão da inicialização, o evento **sistema_ok** conduz para q1 (Pronto). Quando o usuário inicia o treinamento, ocorre a transição para q2 (Demonstração de Movimento). Concluída essa etapa, o evento **fim_demo** leva para q3 (Aguardando). Nesse estado, o jogador poderá iniciar a tentativa por meio do botão apresentado na tela. Seu acionamento produz o evento **botao_iniciar**, permitindo a transição para q4 (Captura), desde que a câmera esteja disponível. Ao concluir o registro do movimento, **fim_captura** conduz para q5 (Análise), onde os dados são processados e avaliados.

Se a execução atender aos critérios estabelecidos, o evento **resultado_ok** leva para q6 (Correto). Caso contrário, **resultado_erro** conduz para q7 (Erro de Movimento), seguido de **emitir_feedback**, responsável pela transição para q8 (Feedback), no qual são apresentadas as orientações corretivas. Após a conclusão do feedback, o fluxo retorna para q2, permitindo uma nova demonstração e tentativa. Quando o movimento é considerado correto, o evento **proximo** permite avançar para o exercício seguinte. A sessão também poderá ser pausada, encerrada ou interrompida por uma falha técnica, conforme as transições previstas na máquina de estados.

1.3 Definição Formal Atualizada

O comportamento do TennisSync continua sendo representado por um modelo de estados finitos determinístico, considerando os estados e eventos responsáveis pelo controle do treinamento. A definição formal foi atualizada para incorporar a substituição do pedal por um botão na interface, mantendo o conjunto de estados e as demais transições estabelecidas na primeira entrega.

A representação formal permanece:

$$\mathbf{M = (Q, \Sigma, \delta, q_o, F)}$$

Em que:

- **Q:** conjunto de estados;
- **Σ:** conjunto dos eventos de entrada;
- **δ:** função de transição;
- **q_0:** estado inicial;
- **F:** conjunto de estados finais.

O conjunto de estados é:

$$\mathbf{Q = \{q0, q1, q2, q3, q4, q5, q6, q7, q8, q9, q10, qE\}}$$

O alfabeto de eventos (Σ) foi atualizado para incorporar **botao_iniciar**, representado pelo símbolo **h**, substituindo o antigo acionamento do pedal. O símbolo **b** permanece associado ao evento **reset**. Para manter uma representação consistente, os símbolos são relacionados aos eventos da seguinte forma:

Símbolo	Evento
o	sistema_ok
i	iniciar
d	fim_demo
h	botao_iniciar
c	fim_captura
k	resultado_ok
e	resultado_erro

f	emitir_feedback
t	fim_feedback
x	proximo
s	pausar
r	retomar
z	encerrar
u	falha
b	reset

O alfabeto atualizado é:

$$\Sigma = \{o, i, d, h, c, k, e, f, t, x, s, r, z, u, b\}$$

O estado inicial permanece:

$$q_0 = q0$$

E o conjunto de estados finais é:

$$F = \{q10\}$$

A principal alteração na função de transição é:

$$\delta(q3, h) = q4$$

Em sua forma descritiva:

$$\delta(q3, botao_iniciar) = q4$$

Essa expressão indica que, ao receber o comando de acionamento do botão no estado q3 (Aguardando), ocorre a transição para q4 (Captura).

Na modelagem adotada, a função de transição é determinística e parcial, pois cada combinação válida de estado e evento possui um único próximo estado, enquanto as combinações não definidas representam comandos não autorizados. Essa estrutura é suficiente para descrever o fluxo de controle do treinamento. Entretanto, na definição formal clássica de um Autômato Finito Determinístico (AFD), a função de transição deve ser total, ou

seja, deve existir uma transição para cada combinação de estado e símbolo do alfabeto. Para obter um AFD completo equivalente, seria possível acrescentar um estado de rejeição, direcionando para ele as combinações não definidas e estabelecendo autotransições nesse estado para todos os eventos. Essa complementação formal preservaria as sequências aceitas e não modificaria as etapas operacionais previstas para o treinamento.

2. Descrição dos Eventos

Os eventos representam as ações, comandos ou condições responsáveis pelas mudanças de estado durante o funcionamento da aplicação. Eles permitem controlar a sequência de treinamento e determinar quais operações deverão ser executadas em cada etapa.

Com a retirada do pedal, o evento associado ao dispositivo físico foi substituído por **botao_iniciar**, correspondente ao acionamento do botão de captura na interface. Os demais eventos foram mantidos conforme a modelagem anterior.

2.1 Relação dos Eventos

Os eventos podem representar ações do jogador, comandos da aplicação ou condições identificadas durante o treinamento.

Na segunda entrega, a principal alteração corresponde ao evento **botao_iniciar**, que substitui o acionamento do pedal e representa o comando realizado pelo jogador por meio de um botão na interface.

- **sistema_ok:** indica que a inicialização foi concluída corretamente.
- **iniciar:** inicia a sessão de treinamento.
- **fim_demo:** indica que a demonstração do movimento terminou.
- **botao_iniciar:** representa o acionamento do botão na tela pelo jogador para iniciar a captura do movimento.
- **fim_captura:** indica que a captura do movimento foi concluída.
- **resultado_ok:** indica que o movimento foi considerado correto de acordo com os critérios definidos.
- **resultado_erro:** indica que foi identificado um erro na execução do movimento.
- **emitir_feedback:** inicia a apresentação das orientações corretivas ao jogador.
- **fim_feedback:** indica que o feedback terminou e que o movimento poderá ser demonstrado novamente.
- **proximo:** faz o treinamento seguir para a demonstração do próximo movimento ou repetição.
- **pausar:** interrompe temporariamente a sessão de treinamento.
- **retomar:** continua a sessão após a pausa, retornando à demonstração do movimento.
- **encerrar:** finaliza a sessão de treinamento.
- **falha:** indica a ocorrência de um problema técnico crítico.
- **reset:** reinicia a aplicação após uma falha, retornando ao estado de inicialização.

2.2 Relação entre Eventos e Transições

Os eventos somente poderão provocar mudanças de estado quando estiverem associados a uma transição prevista no modelo. A tabela apresenta o estado atual, o evento recebido, o próximo estado e as condições ou ações relacionadas à transição.

Estado atual	Evento	Próximo estado	Condição e ação
q0	sistema_ok	q1	Inicialização concluída corretamente; aplicação liberada para treinamento.
q1	iniciar	q2	Comando recebido; inicia a demonstração do movimento.
q1	encerrar	q10	Comando de encerramento recebido; finaliza a sessão.
q2	fim_demo	q3	Demonstração concluída; aguarda o comando do jogador.
q3	botao_iniciar	q4	Botão acionado e câmera disponível; inicia a captura.
q3	pausar	q9	Comando de pausa recebido; interrompe temporariamente a sessão.
q3	encerrar	q10	Comando de encerramento recebido; finaliza o treinamento.
q4	fim_captura	q5	Captura concluída e dados disponíveis; inicia a análise.
q5	resultado_ok	q6	Movimento atende aos critérios de avaliação.
q5	resultado_erro	q7	Movimento apresenta diferenças em relação aos parâmetros esperados.
q6	proximo	q2	Avança para a demonstração do próximo movimento ou repetição.

q6	encerrar	q10	Finaliza o treinamento após o resultado correto.
q7	emitir_feedback	q8	Inicia a apresentação das orientações corretivas.
q8	fim_feedback	q2	Feedback concluído; retorna à demonstração.
q9	retomar	q2	Retoma o treinamento e apresenta novamente o movimento.
q0 a q9	falha	qE	Falha técnica crítica identificada; interrompe as operações afetadas.
qE	reset	q0	Comando de reinicialização recebido; reinicia a aplicação.

A transição de q3 (Aguardando) para q4 (Captura) representa a principal alteração em relação à primeira entrega, pois o acionamento do pedal foi substituído pelo evento **botao_iniciar**, realizado por meio de um botão na interface.

As demais transições permanecem conforme a modelagem anterior, contemplando a análise dos movimentos, a apresentação do feedback, a repetição dos exercícios, a pausa, o encerramento e o tratamento de falhas técnicas. Os critérios indicados na tabela estabelecem os requisitos necessários para autorizar cada operação. Eventos recebidos em estados para os quais não existe uma transição definida não deverão provocar alterações indevidas no funcionamento da aplicação.

3. Regras de Funcionamento

As regras de funcionamento estabelecem as condições que controlam a execução das operações. Seu objetivo é garantir que as etapas do treinamento ocorram na sequência prevista e que comandos inadequados não provoquem comportamentos inesperados.

Essas regras foram organizadas considerando a máquina de estados, os eventos e as funcionalidades descritas na modelagem do projeto.

3.1 Regras de Inicialização

R01 - Disponibilidade dos componentes

O treinamento somente poderá ser iniciado após a conclusão da inicialização e a verificação da disponibilidade dos componentes necessários ao funcionamento da aplicação.

R02 - Tratamento de falhas na inicialização

Caso seja identificada uma falha crítica durante a inicialização, o evento **falha** deverá conduzir para qE (Falha), impedindo o início do treinamento até que seja realizado o procedimento de recuperação por meio de **reset**.

3.2 Regras de Demonstração e Captura

R03 - Demonstração do movimento

Após o início do treinamento, deverá ser apresentado o movimento de referência antes de disponibilizar a captura da tentativa correspondente.

R04 - Início da captura

A captura somente poderá ser iniciada quando a aplicação estiver em q3 (Aguardando) e receber o evento **botao_iniciar**, correspondente ao acionamento do botão disponível na interface pelo jogador, desde que a câmera esteja disponível.

R05 - Captura única por tentativa

Enquanto a aplicação estiver em q4 (Captura), novos comandos de início não deverão gerar outra tentativa simultaneamente.

R06 - Conclusão da captura

A análise somente poderá começar após a conclusão da captura e a disponibilização dos dados necessários ao processamento. Nesse momento, o evento **fim_captura** deverá conduzir de q4 (Captura) para q5 (Análise).

3.3 Regras de Análise e Feedback

R07 - Avaliação do movimento

O movimento capturado deverá ser comparado com os parâmetros definidos para o exercício. O resultado determinará se a execução será classificada como correta ou incorreta.

R08 - Resultado correto

Quando a execução atender aos critérios de avaliação, o resultado deverá ser reconhecido como correto, permitindo o avanço para o próximo movimento ou o encerramento da sessão.

R09 - Resultado incorreto

Quando a tentativa não atender aos critérios estabelecidos, deverá ser identificada como incorreta, encaminhando o jogador para a etapa de feedback.

R10 - Feedback corretivo

O feedback deverá ser apresentado após a identificação de uma execução incorreta, utilizando os recursos visuais e sonoros previstos para orientar o jogador.

R11 - Nova tentativa

Após a conclusão do feedback, o fluxo deverá retornar à demonstração do movimento, permitindo que o jogador realize uma nova tentativa.

3.4 Regras de Pausa, Encerramento e Falhas

R12 - Pausa do treinamento

Na modelagem adotada, a pausa poderá ser solicitada quando a aplicação estiver em q3 (Aguardando), interrompendo temporariamente a sequência de treinamento.

R13 - Retomada do treinamento

Quando a sessão estiver pausada, o comando de retomada deverá conduzir novamente à demonstração do movimento, preservando a sequência prevista.

R14 - Encerramento da sessão

O encerramento poderá ocorrer nos estados q1 (Pronto), q3 (Aguardando) ou q6 (Correto), mediante o evento **encerrar**. Após a transição para q10 (Encerrado), nenhuma nova captura ou análise relacionada à sessão finalizada deverá ser iniciada.

R15 - Tratamento de falhas técnicas

Quando ocorrer uma falha técnica crítica durante o funcionamento, as operações afetadas deverão ser interrompidas e a máquina deverá entrar em qE (Falha). A recuperação ocorrerá por meio do evento **reset**, que retorna à inicialização.

Essas regras estabelecem restrições para cada operação, evitando que comandos sejam executados fora dos estados previstos.

4. Pré e Pós-condições

As pré e pós-condições definem os requisitos necessários para a execução das principais operações do treinamento e os resultados esperados após sua realização.

As pré-condições representam as situações que devem ser verdadeiras antes de uma operação, enquanto as pós-condições descrevem os resultados ou estados esperados após sua execução. A tabela a seguir organiza essas condições de acordo com a máquina de estados desenvolvida.

Operação	Pré-condições	Pós-condições
Inicialização do Sistema	A aplicação deve ter sido acionada ou recebido o evento **reset** após uma falha. Os componentes necessários devem estar disponíveis para verificação.	Se a inicialização for concluída corretamente, **sistema_ok** conduz para q1 (Pronto). Em caso de falha crítica, **falha** conduz para qE (Falha).
Início do Treinamento	A máquina deve estar em q1 (Pronto), com os recursos necessários disponíveis, e receber o comando **iniciar**.	O fluxo passa para q2 (Demonstração de Movimento), disponibilizando o movimento de referência ao jogador.
Início da Captura	A máquina deve estar em q3 (Aguardando), a demonstração deve ter sido concluída, a câmera deve estar disponível e o jogador deve acionar **botao_iniciar**.	O fluxo passa para q4 (Captura), iniciando o registro do movimento pela câmera.
Análise do Movimento	A máquina deve estar em q5 (Análise), com captura concluída, dados disponíveis e parâmetros de referência definidos.	Os dados são processados e o movimento é classificado. **resultado_ok** conduz para q6 (Correto), enquanto **resultado_erro** conduz para q7 (Erro de Movimento). Falhas técnicas críticas conduzem para qE.
Apresentação do Feedback	O movimento deve ter sido classificado como incorreto, a máquina deve estar em q7 (Erro de Movimento) e receber o evento **emitir_feedback**.	O fluxo passa para q8 (Feedback), apresentando as orientações corretivas. Após **fim_feedback**, retorna para q2 (Demonstração de Movimento).
Pausa do Treinamento	A máquina deve estar em q3 (Aguardando) e receber o evento **pausar**.	O fluxo passa para q9 (Pausado), interrompendo temporariamente a sequência de treinamento.
Retomada do Treinamento	A máquina deve estar em q9 (Pausado) e receber o evento **retomar**.	O fluxo retorna para q2 (Demonstração de Movimento), apresentando novamente o movimento antes de uma nova captura.

Encerramento do Treinamento	A máquina deve estar em q1 (Pronto), q3 (Aguardando) ou q6 (Correto) e receber o evento **encerrar**.	O fluxo passa para q10 (Encerrado), finalizando a sessão e impedindo novas capturas ou análises relacionadas ao treinamento encerrado.

O estabelecimento dessas condições permite verificar se cada operação ocorre no estado adequado e se os resultados produzidos correspondem ao comportamento previsto na máquina de estados.

As condições apresentadas representam o comportamento esperado das operações. Em caso de falha técnica crítica, deverá ser realizado o tratamento previsto para qE, evitando que uma operação seja considerada concluída sem produzir o resultado necessário.

5. Modelo Computacional

O modelo computacional adotado para representar o funcionamento do TennisSync é baseado no **Autômato Finito Determinístico (AFD)**. Essa escolha se justifica porque o fluxo de controle do treinamento pode ser descrito por um conjunto finito de estados e eventos, permitindo organizar as etapas da aplicação e determinar as transições autorizadas. A modelagem utiliza uma função de transição determinística e parcial, que pode ser completada formalmente para corresponder à definição clássica de um AFD.

Os modelos computacionais estudados na disciplina apresentam diferentes capacidades de representação. O Autômato Finito utiliza um conjunto limitado de estados e transições, sendo adequado para descrever comportamentos previamente definidos. O Autômato com Pilha possui uma estrutura adicional de memória, utilizada para representar determinadas dependências entre símbolos. Já a Máquina de Turing constitui um modelo computacional mais geral, capaz de representar algoritmos que exigem operações de leitura, escrita e armazenamento de informações.

Na solução desenvolvida, o AFD é adequado porque o treinamento possui etapas bem definidas, como inicialização, demonstração do movimento, espera pelo acionamento do botão, captura, análise, apresentação dos resultados, feedback e encerramento da sessão. Cada etapa corresponde a um estado, enquanto os eventos recebidos determinam as mudanças permitidas entre eles.

O caráter determinístico está associado ao fato de que, para cada combinação de estado e evento permitida, existe um único próximo estado definido. Por exemplo, quando a aplicação está em q3 (Aguardando) e recebe o evento **botao_iniciar**, ocorre a transição para q4 (Captura).

Formalmente:

$$\delta(q3, botao_iniciar) = q4$$

Da mesma forma, o resultado da análise determina as seguintes transições:

$$\delta(q5, resultado_ok) = q6$$

$$\delta(q5, resultado_erro) = q7$$

Essas expressões demonstram como a máquina de estados representa os possíveis caminhos do treinamento, permitindo verificar se os eventos recebidos respeitam as regras estabelecidas.

A utilização de um Autômato com Pilha não é necessária, pois o controle das etapas não exige uma estrutura de memória do tipo pilha nem o reconhecimento de dependências aninhadas. Da mesma forma, embora a Máquina de Turing permita representar processos computacionais mais complexos, sua capacidade não é necessária para descrever as transições entre os estados definidos. É importante destacar que o AFD representa o fluxo de controle da sessão, e não todos os algoritmos executados pela aplicação. A identificação dos pontos corporais pelo MediaPipe, a comparação dos movimentos e o cálculo da pontuação envolvem operações de processamento de dados realizadas por outros componentes computacionais.

Portanto, o **Autômato Finito Determinístico (AFD)** constitui a referência computacional mais adequada para representar o fluxo de controle da aplicação, pois permite organizar o treinamento por meio de estados e transições determinísticas. Embora a modelagem operacional utilize uma função parcial, ela pode ser convertida em um AFD completo equivalente, sem alterar as sequências válidas previstas. Dessa forma, não é necessário recorrer à memória adicional de um Autômato com Pilha ou à capacidade computacional mais ampla de uma Máquina de Turing para representar o controle das sessões.

6. Análise de Decidibilidade

Esta análise busca identificar as decisões de controle do TennisSync e verificar se elas podem ser resolvidas por algoritmos bem definidos. No funcionamento proposto, essas decisões são realizadas por meio da verificação dos estados, eventos e condições estabelecidas, permitindo determinar quais operações são autorizadas durante o treinamento.

6.1 Validação do Início da Captura

Uma das decisões realizadas pela aplicação consiste em verificar se o jogador pode iniciar a captura do movimento.

De acordo com a máquina de estados, o evento **botao_iniciar** somente possui uma transição prevista quando o estado atual é q3 (Aguardando). Além disso, a câmera deve estar disponível para que o movimento possa ser registrado. Assim, o programa pode verificar o estado atual e a disponibilidade da câmera para determinar se a operação pode ser iniciada.

A regra pode ser expressa como:

PodeCapturar(q, c) = 1, se q = q3 e c = 1; 0, caso contrário.

Nessa expressão, q representa o estado atual e c indica a disponibilidade da câmera, sendo 1 para disponível e 0 para indisponível.

O resultado 1 representa a autorização da captura, enquanto o resultado 0 indica que as condições necessárias não foram atendidas.

Essa decisão é determinística, pois, para os mesmos valores de entrada, a regra sempre produz o mesmo resultado. Também é decidível, uma vez que depende de verificações simples que terminam após a comparação das condições estabelecidas.

6.2 Verificação de Transições Permitidas

Outra decisão importante consiste em verificar se determinado evento pode provocar uma mudança de estado.

A máquina de estados define uma função de transição parcial:

$$\delta: Q \times \Sigma \rightharpoonup Q$$

Nessa representação, nem todas as combinações de estado e evento possuem uma transição definida. Para verificar a validade de um comando, a aplicação consulta a função de transição e identifica se existe um próximo estado correspondente.

Por exemplo:

$$\delta(q3, \text{botao_iniciar}) = q4$$

Essa transição é permitida, pois representa o início da captura a partir de q3 (Aguardando).

Por outro lado, não existe uma transição definida para **botao_iniciar** a partir de q5 (Análise). Portanto, caso esse comando seja recebido durante o processamento, não deverá provocar o início de outra captura. Como o conjunto de estados e eventos é finito, a existência de uma transição pode ser verificada por um procedimento que termina após a consulta das regras estabelecidas.

Dessa forma, a validação das transições é uma decisão determinística e decidível, contribuindo para impedir a execução de comandos em estados inadequados.

6.3 Classificação do Movimento

Durante a análise, os dados capturados são comparados com os parâmetros estabelecidos para o exercício. A partir dessa comparação, deve ser produzido um resultado que permita distinguir uma execução correta de uma incorreta. Considerando que o algoritmo de avaliação forneça uma pontuação numérica válida e que exista um limite mínimo previamente definido, a classificação poderá ser realizada por meio de uma comparação.

Seja P a pontuação calculada e L o limite mínimo de aprovação:

Resultado(P) = correto, se P ≥ L; incorreto, se P < L.

Nesse exemplo, a decisão é determinística porque, para uma mesma pontuação e um mesmo limite, o resultado será sempre o mesmo. A comparação numérica também é decidível, considerando entradas válidas e uma representação computacional definida, pois o procedimento termina após verificar se a pontuação atingiu o limite estabelecido.

Essa verificação corresponde à decisão final de classificação do movimento. O processamento realizado pelo MediaPipe e pelo algoritmo de avaliação possui procedimentos próprios, que devem ser considerados separadamente da comparação final. Caso os dados necessários não estejam disponíveis ou apresentem uma falha crítica, a situação deverá ser tratada como um problema técnico, e não classificada automaticamente como uma execução incorreta.

6.4 Verificação da Possibilidade de Retomada

Outra decisão consiste em identificar quando uma sessão pausada pode ser retomada.

Na modelagem definida, a retomada ocorre quando o estado atual é q9 (Pausado) e o evento recebido é **retomar**.

A transição é representada por:

δ(q9, retomar) = q2

Essa expressão indica que, ao receber o comando de retomada em q9, o fluxo retorna para q2 (Demonstração de Movimento), permitindo que o jogador visualize novamente o movimento de referência. A decisão consiste em verificar se o estado atual corresponde a q9 e se o evento recebido é **retomar**.

Como essa operação envolve a comparação de valores pertencentes a conjuntos finitos, pode ser realizada por um procedimento determinístico que termina e fornece uma resposta definida. Portanto, a verificação da possibilidade de retomada também é considerada decidível.

6.5 Reconhecimento de Sequências Válidas

O Autômato Finito também permite verificar se determinada sequência de eventos corresponde a uma execução válida. Para isso, cada evento é processado de acordo com a função de transição. Uma sequência é aceita quando parte do estado inicial q0 e termina no estado final q10, respeitando todas as transições definidas.

A condição de aceitação pode ser representada por:

$$L(M) = \{w \in \Sigma^* \mid \delta^*(q0, w) = q10\}$$

Em que:

- **w:** representa uma sequência finita de eventos.
- **Σ*:** corresponde ao conjunto de todas as sequências finitas formadas pelos eventos.
- **δ*:** representa a aplicação sucessiva da função de transição sobre uma sequência.
- **q10:** representa o estado final do treinamento.

Como exemplo, considere uma sessão em que o jogador executa corretamente o movimento na primeira tentativa. A sequência de eventos pode ser representada por:

$$\mathbf{w_1 = o \cdot i \cdot d \cdot h \cdot c \cdot k \cdot z}$$

Nessa representação, os símbolos correspondem aos seguintes eventos:

- **o:** sistema_ok;
- **i:** iniciar;
- **d:** fim_demo;
- **h:** botao_iniciar;
- **c:** fim_captura;
- **k:** resultado_ok;
- **z:** encerrar.

A sequência representa a inicialização correta, o início do treinamento, a conclusão da demonstração, o acionamento do botão na interface, a finalização da captura, a identificação de um movimento correto e o encerramento da sessão.

Por outro lado, a sequência $\mathbf{w_2 = o \cdot i \cdot h \cdot c \cdot k \cdot z}$ é inválida, pois tenta iniciar a captura antes da conclusão da demonstração. Nesse caso, o evento **botao_iniciar** não possui uma transição definida a partir de q2 (Demonstração de Movimento). Como a máquina possui um conjunto finito de estados e cada evento de uma sequência finita pode ser processado pela função de transição, a verificação sempre termina. O procedimento pode identificar uma transição inválida ou concluir o processamento, verificando se o estado alcançado pertence ao conjunto de estados finais.

Portanto, o reconhecimento das sequências aceitas pela máquina de estados é um problema decidível, pois existe um procedimento determinístico capaz de verificar cada evento e fornecer uma resposta definida.

6.6 Considerações sobre Determinismo e Decidibilidade

As decisões de controle são baseadas em estados, eventos e critérios definidos previamente. Para as operações analisadas, é possível estabelecer procedimentos que produzem respostas objetivas e terminam sua execução.

O determinismo garante que uma mesma entrada, nas mesmas condições, siga as mesmas regras de processamento. A decidibilidade, por sua vez, está relacionada à existência de um algoritmo que termine e responda corretamente ao problema definido. Na solução proposta, essas propriedades podem ser observadas na validação do início da captura, na verificação das transições permitidas, na classificação dos movimentos, na retomada do treinamento e no reconhecimento de sequências válidas. Dessa maneira, a modelagem formal auxilia na organização do funcionamento da aplicação e permite identificar quais decisões podem ser verificadas computacionalmente, contribuindo para a consistência das regras estabelecidas e para o comportamento esperado.

Considerações Finais

A revisão da modelagem formal alinhou a máquina de estados ao estágio atual de desenvolvimento do projeto, destacando-se a substituição do acionamento por pedal por um botão na interface. A estrutura da primeira entrega foi mantida, preservando os estados e atualizando os eventos relacionados ao funcionamento do treinamento. A definição das regras e condições estabelece os limites de cada operação, enquanto a escolha do modelo de estados finitos determinístico permite representar as principais etapas da sessão. A análise de decidibilidade demonstra que diferentes decisões de controle podem ser verificadas por procedimentos computacionais definidos. Com isso, a modelagem se consolida como uma base técnica segura para guiar a implementação e garantir a validação correta do TennisSync na prática.